STEPLA-1 Control Unit Design Specification Manual

Project Name: STEPLA-1 CPU
Module: Control Unit (CU)
Author(s): Ahmed Khan, Muhammed Khizar, Shaheer Afzal
Document Status: Under Development
Version: 2.4.2
Date: 01/01/2026
Architecture Type: Harvard Architecture
Control Type: Hardwired Control Unit

Document Version History

Version	Date	Author(s)	Description
1.0.0	Jan 2026	Ahmad	ROM based CU
2.0.0	Feb 2026	Ahmad, M.Khizar, Shaheer.A	Discrete Logic Gate based CU
2.1.0	Feb 2026	Ahmad, M.Khizar, Shaheer	Added GETPC, JMP Rd Instructions.
2.2.0	Feb 2026	Ahmad, M.Khizar, Shaheer.A	Changes in Add and Subtract
2.2.1	Feb 2026	Ahmad	Revision
2.2.2	Feb 2026	Ahmad, Khizar	Revision and minor rewordings
2.3.0	Feb 2026	Ahmad, M.Khizar, Shaheer.A	Changes to ISA Removed LDI, Added Load Immediate and Store Immediate
2.4.0	Feb 2026	Ahmad, M.Khizer	Early Exit branching And Cycle Compression
2.4.1	Feb 2026	Ahmad, M.Khizer, Shaheer.A	Revisions, Latency profiles
2.4.2	Feb 2026	Ahmad	Signal Stabilization and BUS Termination

Overview **5**
1. Control Unit Overview 6
1.1 Role of the Control Unit 6
1.2 Control Philosophy 6
1.2.1 Step Counter 7
1.2.2 Logical Working 7
1.2.3 Programmable Logic Array 7
1.2.4 Limitations of PLA 7
1.2.5 Hierarchical Control Logic in STEPLA-1 8
1.3: State Transition & Synchronous Vectoring **8**
1.3.1: Phase Differentiation (Cold Boot vs. Runtime) 8
1.3.2: Synchronous "Load-to-One" Logic 9
2. Control Unit Block Diagram **10**
3. Internal Components 10
3.1 Opcode Register 10
3.2 Address Register 11
3.3 Opcode Decoder 11
3.4 Step Counter 11
3.5 Step Decoder 11
4. Instruction Format 13
4.1 Opcode Field (Bits 7–4) 13
4.2 Register Address Fields (Bits 3–0) 13
4.3 Operand Role Determination 14
5. Instruction Set **14**
5.1 Purpose of the Instruction Set 15
5.1.1 Software and Hardware 15
5.2 Instruction set supported by STEPLA-1 15
5.3 Instruction Encoding Rules 16
5.3.1 Data Transfer Instructions 17
5.3.2 Arithmetic Instructions 17
5.3.3 Control Flow Instructions 18
5.3.4 Output 18
5.4 Operand Address Encoding 19
6. Control Signals **19**
6.1 Control Signals Of STEPLA-1 20
6.1.1 Special Control Signals 22
7. Execution Model **23**
7.1 Instruction Cycle Overview 24
7.1.1 Fetch Cycle 24
7.1.2 End of Instruction 24

7.1.3 Decoding Cycle..... 25
7.2 Data Transfer Instructions..... 25
7.2.1 MOV Instruction..... 25
7.2.3 MOVI Instruction..... 25
7.2.4 STRD Instruction..... 25
7.2.5 LDIM Instruction..... 26
7.2.6 LDD Instruction..... 26
7.3 Control Flow Instructions..... 26
7.3.1 NOP Instruction..... 26
7.3.2 HLT Instruction..... 27
7.3.3 JMP Instruction..... 27
7.3.4 JP Instruction..... 27
7.3.5 JZ, JC Instructions..... 27
7.3.5.1 Conditional Branching In JZ and JC..... 27
The "Fibonacci" Efficiency Case Study..... 28
Hardware Implementation Note: Conditional Path Logic..... 28
7.3.6 GETPC..... 29
Functional Purpose..... 29
7.4 Arithmetic Instructions..... 29
7.4.1 ADD..... 29
7.4.2 SUB..... 29
7.5 Output..... 30
7.5.1 OUT..... 30
8. Control Unit Sequencing and Clock Phase Synchronization..... 30
8.1 Phase Separation and Propagation Delay..... 30
8.2 The T0 Null State and Bootloader Handover..... 31
8.2.1 Timing Diagram for BCU handover (delay of 50ns included)..... 31
8.3 Instruction Fetch and Latching (T1–T3)..... 32
9. Bootstrap Control Unit (BCU) Specification..... 33
9.1 Functional Overview..... 33
9.2 Bus Arbitration and CPU Suspension..... 33
9.3 The Dual-Cycle DMA Protocol..... 33
Phase A: Address Latching (State 0)..... 33
Phase B: Data Transfer (State 1)..... 34
9.4 Release and Synchronization (The T0 Transition)..... 34
9.5 Technical Constraints..... 34
10. Control Logic Generation..... 35
10.1 Core Principle: The Matrix Decoder..... 35
Example: Load Accumulator (LDA) Execution..... 35
10.2 Signal Consolidation (The OR-Bus)..... 35
11. Timing Analysis & Operational Limits..... 35
11.1 Real-World Timing Constraints..... 36
11.2 Component Latency Profiles..... 36
12. Signal Conditioning and Bus Termination..... 36

12.1 Active-Low Logic & Capacitive Displacement..... 36
12.2 Schmitt Trigger Conditioning (74HC14)..... 37
12.3 Bus Pull-up Networks & Passive Termination..... 37
13. Design Decisions..... 38
14. Verification Plan..... 38
15. Known Limitations..... 39
16. Future Architectural RoadMap (v3.0.0)..... 39
16.1 Multi-Cycle Instruction Fetching..... 39
16.2 Expansion of Register File and Opcode Space..... 39
16.3 Hardware-Level Stack Management..... 39
16.4 Asynchronous Dual Control Unit System..... 40
16.5 Advanced Interrupt Vectoring and Priority..... 40
16.6 Pre-Fetch Buffer and Cycle Reduction..... 40
17. Inspirations & Learning Material..... 41
16.1 Architectural Influences..... 41
16.2 Recommended Reading..... 41

Overview

*This document presents the design specification for the **STEPLA-1**, an 8-bit general-purpose computer architecture developed with a focus on educational transparency and hardware-level logic clarity. Diverging from modern microcoded architectures, the STEPLA-1 utilizes a **hardwired control unit** to execute a custom Instruction Set Architecture (ISA) through discrete logic gates and hierarchical decoding.*

*The architecture features a dual-phase clocking system to mitigate race conditions, an 8-bit data bus, and a specialized **Bootstrap Control Unit (BCU)** designed to manage system initialization and cold-boot transitions. The control unit is decomposed into four primary sub-modules: the Pulse Address Selector, Step Decoder, Opcode Decoder, and Control Matrix, ensuring a modular and scalable design.*

*Initial verification via **Logisim-Evolution** confirms the successful execution of arithmetic, logical, and control-flow instructions with a consistent 0.15 to 0.5 Instructions Per Cycle (IPC) throughput.*

1. Control Unit Overview

1.1 Role of the Control Unit

The Control Unit coordinates the processor operations by generating control signals derived from decoded instructions and timing states. It follows given instructions and decodes them into control signals that are used to manipulate different parts of the processor to either read, write or process data.

1.2 Control Philosophy

The philosophy behind the control unit is to run multiple ready made or pre-defined instructions under a single input instruction, i.e. For an instruction to move data between registers mov a,b Control unit has to move the data from **register "b" to register "a"** without causing conflicts at the bus and thus it does by activating control signals in microsteps, controlled by step counter present in it.

1.2.1 Step Counter

Step Counter or **micro-step counter** is a 3-4 bit counter that increments after every clock/machine cycle, tells the **Control Unit**, which step of an instruction it is at and thus this leads to specific control signals activating at each micro-step of an instruction.

1.2.2 Logical Working

The control unit of **STEPLA-1** is specifically **hard wired**, meaning instead of using a ROM or a premade and pre-defined block it uses a logic purely made for controlling the whole processor. This decision was purely made on the basis of **more transparency** and on the goal of understanding logic.
The **Control Unit** follows a **PLA-inspired combinational logic** organization implemented using **discrete logic gates**, that implements gates to respond to specific inputs and toggle a signal output. Such that, the instruction and the stepcounter are purely responsible for changing a control signal to high on a specific input case. The name **STEPLA-1** was used to define this feature of the CPU.

1.2.3 Programmable Logic Array

A **Programmable Logic Array (PLA)** is a structured combinational logic architecture in which input variables and their complements are applied to a programmable AND array to

generate product terms, which are then selectively summed through a programmable OR array to produce multiple output logic functions.
This property of PLA makes it one of the best candidates for CU internals as we can now control specific outputs with specific inputs and those outputs are purely dependent on "Op-code + Step" format.

1.2.4 Limitations of PLA

Although Programmable Logic Arrays (PLAs) provide a structured and intuitive method for implementing combinational logic, they exhibit several practical limitations as system complexity increases.

The primary limitation of a PLA is the **rapid growth in hardware complexity** with increasing numbers of inputs and control conditions. Since each unique combination of inputs requires a dedicated product term in the AND plane, the number of required connections grows exponentially as additional variables are introduced. This phenomenon is commonly referred to as **logic explosion**.
A 4 input system can have as many as 16 logical combinations whereas an 8 input system has 256 logical combinations.

For a control unit, inputs may include:

1. Opcode bits(4 bits)
2. Timing step signals(3 bits)
3. Status flags (Carry Flag, Zero Flag 2-bits)
4. Instruction modes or control states

As these inputs increase, a single flat PLA implementation becomes really difficult and error prone in designing, it is hardware intensive due to which we get a **propagation delay** (delay that occurs for a logic unit to change its state), making the response of the **Control Unit** slower. Another problem we may face can be modifications, as it can get impossible to modify these Arrays.

1.2.5 Hierarchical Control Logic in STEPLA-1

To address these limitations, the STEPLA-1 architecture adopts a **hierarchical decoding approach** instead of a single monolithic PLA.

Rather than decoding all control conditions simultaneously, the system divides the logic into multiple structured stages:

1. **Opcode Decoder:** Determines the current instruction class.
2. **Step Decoder:** Identifies the execution phase of the instruction cycle.
3. **Flag Registers:** Provide conditional execution inputs.

4. **Local Logic Combination:** Generates control signals only where required.

This hierarchical organization provides several **advantages** including **reduced gate count, simplified boolean expressions, higher clock speeds, enhanced architecture clarity** and last but not least it made debugging a lot easier.
By decomposing the control logic into smaller functional blocks, **STEPLA-1** achieves behavior equivalent to a PLA while avoiding exponential growth in complexity.

1.3: State Transition & Synchronous Vectoring

1.3.1: Phase Differentiation (Cold Boot vs. Runtime)

The STEPLA-1 architecture differentiates between the **Initialization Phase** and the **Execution Phase**.

- **State T_0 (Cold Boot):** This state is strictly reserved for hardware stabilization following a **Fresh Start** or system reset via the BCU. It acts as a safety buffer for bus arbitration as well as it helps stabilise MAR, since it still contains the last address accessed by the BCU.
- **State T_1 (Runtime Target):** During active program execution, the system bypasses T_0. All instructions now **vector** directly back to T_1 for the subsequent Fetch cycle, ensuring the instruction stream remains unbroken.

1.3.2: Synchronous "Load-to-One" Logic

To eliminate the "Null Cycle" penalty found in v2.3.0, the asynchronous **Reset** signal has been replaced by a **Synchronous Load** operation.

Instead of toggling a global reset (which ignores the clock and risks cutting off data writes), the Control Unit now pulses the **Step Counter's LOAD pin** during the final execution step of every instruction.

- **Merging Cycles:** This "Load" signal is synchronized with the final **Write** pulse of the current instruction.
- **Clock-Edge Handover:** Because the Step Counter is a synchronous device, it only updates to 1 (T_1) on the **Rising Edge** of the clock. This ensures the current instruction has the full clock duration to complete its work before the counter snaps to the next Fetch.

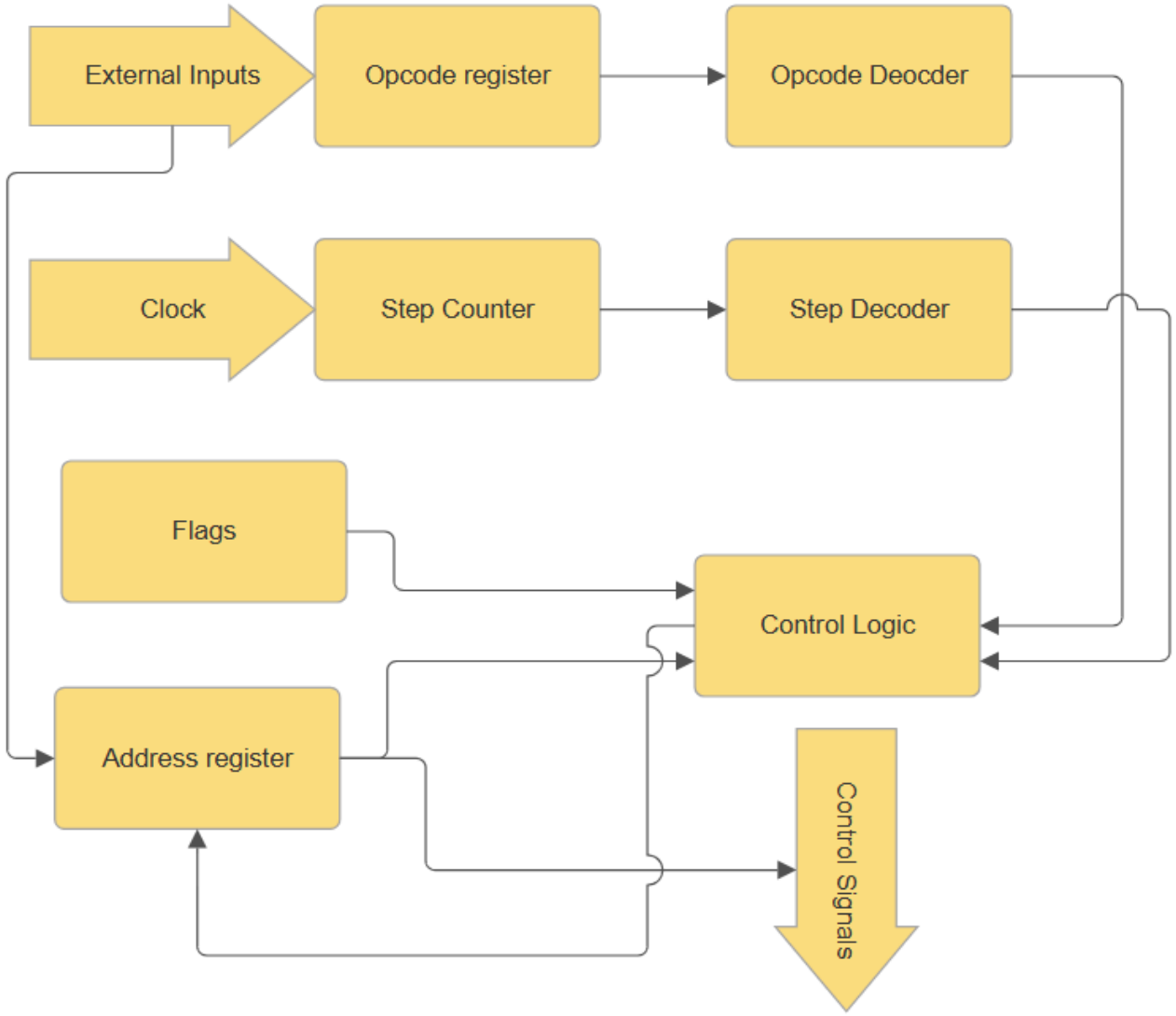
This helps in creating an almost continuous system of fetch, decode and execute cycles. Increasing the efficiency by almost 33%.

Note: Refer to sections 8 and 8.2 for Boot Logic and T_0 null state.

---

2. Control Unit Block Diagram

Figure



---

3. Internal Components

3.1 Opcode Register

Purpose: Stores current instruction opcode. From the RAM obtained in a previous fetch cycle. It has a bit width of 4 bits.

3.2 Address Register

Can store Addresses of **Source** and **destination** registers, that data will be later stored in them. Has a width of 4 bits, 2 bits for each register.

3.3 Opcode Decoder

Opcode decoder takes the value stored at Opcode register and converts it into a single hot-line output that can tell which instruction is being called.

Example:

Opcode Register Output	Decoder Output
0x0	NOP (maps to line 1)
0x1	HLT (maps to line 2)

3.4 Step Counter

The Step Counter is a **clock-driven sequential counter** responsible for tracking the current execution phase of an instruction within the Control Unit. It generates timing states that sequence the micro-operations required to complete an instruction cycle.

Each clock pulse advances the counter to the next step, allowing the Control Unit to activate the appropriate control signals for fetch, decode, execute, and write-back operations.

The initial timing state always corresponds to the **instruction fetch phase**, ensuring that every instruction execution begins by retrieving the next instruction from instruction memory.

3.5 Step Decoder

Converts **step counter** output into **timing signals**. It is to be a single wire driven system that will map to a single output that is capable of enabling the CU logic control to know that at which step it is of a specific instruction.

Example:

T_1 = Fetch phase
T_2 = Decode phase
T_3 = Execute phase
T_n = Counter reset

---

4. Instruction Format

The STEPLA-1 processor uses an **8-bit fixed-length instruction format** interpreted by the Instruction Decoder within the Control Unit.

Each instruction byte is divided into fields as follows:

Bit range	Field Name	Description
7–4	Opcode	Specifies the operation to be executed
3–2	Register Address 1	First register operand
1–0	Register Address 2	Second register operand

4.1 Opcode Field (Bits 7–4)

The **opcode** field determines the instruction type and execution behavior.
Opcodes are represented in:

- Binary format (e.g., `0001`)
- Hexadecimal format (e.g., `0x1`)

The opcode controls:

- Control signal generation
- Execution sequence
- Interpretation of operand fields

4.2 Register Address Fields (Bits 3–0)

Bits **3–0** specify register operands used during instruction execution.

- Bits **3–2** represent **Register Address-A** *→ 1*
- Bits **1–0** represent **Register Address-B** *→ 2*

The interpretation of these operands (source or destination) is determined by the opcode.

4.3 Operand Role Determination

The Control Unit assigns **source** and **destination** roles dynamically based on the opcode definition.

Therefore:

- The same bit fields may represent different operand roles depending on instruction type.
- This allows efficient encoding while maintaining a compact instruction size.

---

5. Instruction Set

The **Instruction Set** defines the complete collection of operations supported by the STEPLA-1 processor. Each instruction is represented by an **opcode**, which corresponds to a specific binary bit pattern interpreted by the Control Unit.

The opcode determines the operation to be performed by directing the Control Unit to generate the required sequence of control signals during instruction execution.

5.1 Purpose of the Instruction Set

5.1.1 Software and Hardware

The **instruction set** serves as the fundamental interface between software and hardware. Each instruction is represented as a sequence of binary bits that specifies the operation to be executed by the Control Unit. These binary encodings allow software to directly influence processor behavior through well-defined hardware actions.

Programs written by developers are not executed directly in high-level programming languages. Instead, compilers designed for specific processor architectures translate high-level code into lower-level representations compatible with the target system. One of the primary intermediate representations is assembly language, where human-readable mnemonics correspond directly to processor instructions.

Assembly programs are then translated by an assembler into binary machine instructions understood by the processor. As a result, multiple layers of software abstraction ultimately resolve into the instruction set, making it the lowest-level contract between software execution and hardware implementation.

By defining fixed opcode meanings, programs can reliably control processor behavior.

5.2 Instruction set supported by STEPLA-1

Following is the table of instruction set that is supported with their binary representations as well as hex representations, source and destination as well as, other minor things that are mentioned.

Opcode (Bin)	Opcode (Hex)	Mnemonic	Operands	Operation	Description
0000	0x0	NOP	-	-	Processor proceeds to next instruction
0001	0x1	HLT	-	-	Processor halts the clock and stop all operations
0010	0x2	Add	Rd, RS	Rd ← Rs+Rd	Add contents of Rd and Rs and store them in Rd
0011	0x3	Sub	Rd, Rs	Rd ← Rd-Rs	Subtract value at accumulator B from A and store at Rd
0100	0x4	MOV	Rd, Rs	Rd ← Rs	Move contents of Rs to Rd
0101	0x5	MOVI	Rd	Rd ← RAM	Move contents of RAM at [current_addr+1] to Rd
0110	0x6	STRD	Rs,Rd	RAM[Rd] ← Rs	Move Value at Rs to DRAM at address stored in Rd
0111	0x7	OUT	Rs	Output ← Rs	Show the value of Rs at Output
1000	0x8	JMP	-	PC ← addr	Modify PC to a new value
1001	0x9	JZ	-	PC ← addr	Modify PC to a new value if flag zero is high
1010	0xA	JC	-	PC ← addr	Modify PC to new value if flag carry is high
1011	0xB	LDIM	Rd	Rd ←DRAM [IRAM[PC +1]]	Load value at DRAM[addr] to Rd Where addr is IRAM[PC+1]
1100	0xC	LDD	Rs,Rd	Rs ← RAM[Rd]	Load from DataRAM[Rd] to Rs
1101	0xD	GETPC	Rd	Rd ← PC	Load Current PC value to Rd
1110	0xE	JP	Rd	PC ← Rd	Load value stored at register Rd to PC
1111	0xF	STIM	Rs	DRAM [IRAM[PC +1]] ← Rs	Store value from Rs to DRAM at address IRAM[PC+1]

5.3 Instruction Encoding Rules

The STEPLA-1 processor uses an **8-bit instruction format**. Each instruction is divided into two fields, the **upper four bits (bits 7–4)** represent the **opcode**, which specifies the operation to be executed, the **lower four bits (bits 3–0)** contain **operand information**, such as register selection, addressing mode, or memory selection bits, as interpreted by the Control Unit.

5.3.1 Data Transfer Instructions

Data transfer instructions move data between registers, accumulators, and memory units. Remember Data RAM can be written and read as well, but Instruction RAM can only be read.

Opcode	Mnemonic	Description
0x4	MOV	Move values between registers
0x5	MOVI	Move value stored at immediate address to Rd
0x6	STRD	Store value from Rs, to DRAM
0xB	LDIM	Load value at DRAM[addr] to Rd Where addr is IRAM[PC+1]
0xC	LDD	Load value from DRAM at address stored in Rs to Rd
0xF	STIM	Store value from Rs to DRAM at address IRAM[PC+1]

MOV instruction at 0x4 requires Source register (2 bits) as well as Destination (2 bits) in order MOV Rd,Rs (i.e, 01000001 will be value stored in Reg ‘B’ to Reg ‘A’), whereas **MOVI** only need Rd and it will move the data at RAM on immediate address to Destination register (i.e, 010100xx will be move value stored at RAM[PC+1] to **Register A**).
STRD, LDD have Rs and Rd, Rd is the address of RAM and Rs is the value that will be stored. LDIM, STIM require only one address Rs, Rd and an immediate address present in IRAM. The Rd and Rs fields both map to **Register Address Pointer A**.

5.3.2 Arithmetic Instructions

The main arithmetic instructions of **STEPPLA-1** are **ADD** and **SUB** with Opcodes **0x2** and **0x3** respectively. They both require a destination address as well as a source address. Whereas, the values to be processed are stored at Accumulator A and B.

5.3.3 Control Flow Instructions

Control Flow Instructions modify the program execution order.

Opcode	Mnemonic	Operands	Operation	Description
0x8	JMP	-	PC ← addr	Modify PC to a new value
0x9	JZ	-	PC ← addr	Modify PC to a new value if flag zero is high
0xA	JC	-	PC ← addr	Modify PC to new value if flag carry is active
0x0	NOP	-	-	Proceed the PC
0x1	HLT	-	-	Halt
0xE	JP	Rd	PC ← Rd	Move contents of Rd to PC
0xD	GETPC	Rd	Rd ← PC	Stored value of PC to register

JMP is unconditional jump instruction that will modify the **PC** to immediate value stored at RAM and will not require any more arguments (i.e, 1000xxxx to jump to a new value of PC). **JC** and **JZ** are conditional and require **Carry Flag** and **Zero Flag** respectively to be true to proceed with a normal **JMP** instruction. **NOP** will Proceed the **Program Counter** to a next value, **the reason NOP exists at 0x0 is a safety measure**, in case an address at RAM is unintentionally left blank the processor will not lose its flow or data. In case of Instruction changes NOP can be used so the determined addresses are not changed, it is used as a white space. **HLT** can be used to stop the processor indefinitely.

5.3.4 Output

Output or **OUT** at **0x7**, is responsible for showing data at any register at Output display. It requires Rs only and will show the value at Rs in Metric Form using a Bin to Hex display decoder. The input will be as follows, 011100xx will show the value stored at register A

5.4 Operand Address Encoding

In STEPLA-1 instructions, the lower four bits of the instruction word specify register addressing information interpreted by the Control Unit as follows:

Refer to *1* which represents the **destination register**.

Refer to *2* which represents the **source register**.

Unless explicitly overridden by an opcode definition, instruction execution follows the convention:

Register Address B → Source
Register Address A → Destination

This convention allows simultaneous read and write operations during a single execution step, where the source register drives the system bus and the destination register latches the resulting value.

---

6. Control Signals

Control signals are binary electrical signals generated by the Control Unit to coordinate and regulate the operation of all datapath components within the processor. These signals determine **when** and **how** individual hardware elements such as registers, memory units, arithmetic logic units (ALU), buses, and input/output interfaces perform their operations.

Each control signal enables, disables, selects, or triggers a specific hardware action during a particular execution step of an instruction. By activating different combinations of control signals across sequential timing states, the Control Unit decomposes an instruction into a series of micro-operations that together implement the required functionality.

Control signals may perform functions including, but not limited to:

1. Enabling register read or write operations,
2. Selecting ALU operations,
3. Controlling memory read or write access,
4. Incrementing or loading the Program Counter,
5. Selecting data paths through multiplexers,
6. Updating processor status flags.

In STEPLA-1, control signals are generated through opcode decoding combined with timing signals produced by the Step Counter, ensuring deterministic execution of each instruction.

6.1 Control Signals Of STEPLA-1

There are approximately 20 control signals in STEPLA-1 that when combined with PLA in control unit can form specific sequences of activations that help it progress with any given instruction. Some signals are internal whereas 8 special signals exist (seperate). Given in Table:-

Signal Name	Type	Target Component	Function Description
PC_OUT	Enable	Program Counter	Program Counter drives BUS
PC_INC	Pulse	Program Counter	Increments the Program Counter on the next clock cycle
PC_RESET	Pulse	Program Counter	Resets the Program Counter

Signal Name	Type	Target Component	Function Description
PC_WRITE	Write/Load	Program Counter	Writes the data from BUS into program counter
REG_RD_A *(Internal)*	Enable	Internal Address(Pointer) Register	Activates the read line of the register specified by Address Field **A**, allowing its contents to drive the system bus.
REG_RD_B *(Internal)*	Enable	Internal Address(Pointer) Register	Activates the read line of the register specified by Address Field **B**, allowing its contents to drive the system bus.
REG_WR_A *(Internal)*	Write/Load	Internal Address(Pointer) Register	Activates the write line of the register specified by Address Field **A**, allowing it to take in value at BUS
REG_WR_B *(Internal)*	Write/Load	Internal Address(Pointer) Register	Activates the write line of the register specified by Address Field **B**, allowing it to take in value at BUS
IMEM_RD	Pulse	Instruction RAM	Instruction RAM drives the BUS
MAR_IN	Enable	Memory Address Register	Enables input line of memory address register

Signal Name	Type	Target Component	Function Description
DMAR_IN	Enable	Data Mem Addr Reg	Enables input line Data mem Addr Reg
DMEM_RD	Enable	Data RAM	Data RAM drives the BUS
DMEM_WR	Write/Load	Data RAM	Enables write line of Data RAM
OP_REG_EN *(Internal)*	Write/Load	Opcode Register	Enables input line of Opcode Register
IAR_EN *(Internal)*	Write/Load	Internal Address Register	Enables input line of Internal Address Register
STP_RST *(Internal)*	Pulse	Step Counter	Reset Step Counter
ALU_SUB	Control	ALU	Enable ALU Subtractor Line
ALU_OUT_EN	Enable	ALU	ALU drives the BUS
FL_EN	Write/Load	Flag register(2-bits)	Enable Flag register Load Line
HALT	Pulse	Clock	Halts the Clock
OUTPUT_REG_EN	Enable	Output Register	Load BUS to Output

6.1.1 Special Control Signals

Following are special control Signals, these are indirectly influenced by the Control Unit. These are register control lines controlled by internal multiplexers and Internal Address Reg.

Signal Name	Type	Target Component	Description
A_RD_EN	Enable	Register A	Register A drives the BUS
B_RD_EN	Enable	Register B	Register B drives the BUS
C_RD_EN	Enable	Register C	Register C drives the BUS
D_RD_EN	Enable	Register D	Register D drives the BUS
A_WR_EN	Write/Load	Register A	Register A stores the value on BUS
B_WR_EN	Write/Load	Register B	Register A stores the value on BUS
C_WR_EN	Write/Load	Register C	Register A stores the value on BUS
D_WR_EN	Write/Load	Register D	Register A stores the value on BUS

7. Execution Model

The STEPLA-1 processor executes instructions using a sequential micro-operation model controlled by a clock-driven Control Unit. Each instruction is divided into a fixed sequence of execution phases governed by the Step Counter.

The execution of any instruction consists of two primary stages:

1. Instruction Fetch
2. Instruction Execute

The fetch phase is identical for all instructions, while the execute phase varies depending on the decoded opcode.

7.1 Instruction Cycle Overview

Every instruction progresses through timing states that determine the **sequence of control signal activation** denoted by T_0 , $T_1 \ldots T_7$. STEPLA-1 uses a 3-bit counter that progresses **7 steps** in one instruction cycle. The second **Time** cycle is always reserved for **fetch operation**, That is primarily used to fetch data stored in **IRAM** to be decoded by Control Unit. Every instruction exists prematurely, after the sought Instruction is complete, using **STP_RST** signal counter is reset, this is used to save CPU cycles instead of waiting for Step Counter to reset itself, CU overrides it and proceeds to the next instruction.

7.1.1 Fetch Cycle

The fetch cycle does not depend on Opcode input, instead it looks if Step Counter has started from T_1, the CU then proceeds to enable output of Program Counter to the BUS and enable Input Line of IMAR, once the address is stored into MAR; CU proceeds to read the IRAM and store the data into Opcode Register and Its Internal Address Register. Program Counter increments one step.

Address Placement:

$T_1 \rightarrow$ P_OUT = 1; MAR_IN = 1

Instruction Read:

$T_2 \rightarrow$ P_INC = 1; IMEM_RD = 1; OP_REG_EN = 1; IAR_EN = 1

7.1.2 End of Instruction

End of Instruction marks the end. It just resets the Step counter to fetch the next instruction. It can be the last of any instruction integrating reset in that same step.
T_n → STP_RST = 1

7.1.3 Decoding Cycle

Decoding Cycle consists of instructions ranging from T_3 till T_n, where the fetched Opcode is executed in pre-defined instructions till End of Instruction.

7.2 Data Transfer Instructions

Data transfer instructions are used to transfer data from one memory module of STEPLA-1 to another, these Instructions include:-

7.2.1 MOV Instruction

MOV instruction **(Opcode 0x4)**, moves data from one register to another, requiring both addresses in form MOV Rd, Rs (Rd → Destination, Rs → Source).

T_3 → REG_A_WR = 1; REG_B_RD = 1; STP_RST = 1
The register selected by the source address enables its output onto the system bus, while the destination register simultaneously latches the bus value. After completion of the transfer, the Step Counter is reset to begin the next instruction fetch cycle.

7.2.3 MOVI Instruction

MOVI instruction **(Opcode 0x5)**, moves data from Instruction RAM into the destination Register. The main reason it is used is to move new data from RAM to registers. LDI cannot be used to achieve similar results as it requires a full address present at one of the registers. MOVI moves the data at RAM[address + 1] directly to the destination register, MOVI Rd.

T_3 →PC_OUT = 1; MAR_IN = 1
T_4 → PC_INC = 1;IMEM_RD = 1; REG_A_WR = 1; STP_RST = 1

Note that the processor follows RISC flow, PC is incremented in advance during the fetch cycle. So to read immediate value we only need to put the PC on MAR and increment it later.

7.2.4 STRD Instruction

STRD instruction **(Opcode 0x6)**, stores data to DRAM. It requires a **Destination** address that has the address of where to store the data, and a **Source** address of what data to be moved to the concerned address. Instruction can be given as STRD Rd,Rs.

T_3 → REG_A_RD = 1; DMAR_IN = 1
T_4 → DMEM_WR = 1; REG_B_RD = 1; STP_RST = 1

7.2.5 LDIM Instruction

LDIM instruction **(Opcode 0xB)** is used to fetch data from DRAM from addresses stored at the next immediate value of the address the instruction is present on, and store the value to Destination Register

T_3 → PC_OUT = 1; IMAR_IN = 1
T_4 → IMEM_RD = 1; DMAR_IN = 1; PC_INC = 1
T_5 → DMEM_RD = 1; REG_A_WR = 1; STP_RST = 1

7.2.6 LDD Instruction

LDD instruction **(Opcode 0xC)** is used to fetch data from DRAM from addresses stored at any of four registers and then saved into the source register or Address B.

T_3 → REG_A_RD = 1; DMAR_IN = 1
T_4 → DMEM_RD = 1; REG_B_WR = 1; STP_RST = 1

7.2.4 STIM Instruction

STIM instruction **(Opcode 0xF)** implements fetching data from DRAM from addresses stored at IRAM on the next immediate value of the PC.

T_3 → PC_OUT = 1; IMAR_IN = 1
T_4 → IMEM_RD = 1; DMAR_IN = 1; PC_INC = 1
T_5 → DMEM_WR = 1; REG_A_RD = 1; STP_RST = 1

7.3 Control Flow Instructions

These instructions are used to control the state of the Processor,

7.3.1 NOP Instruction

NOP instruction **(Opcode 0x0)** is just used to pass a cycle on wait mode. The processor does nothing during this instruction cycle's lifetime.

$T_3 \rightarrow$ STP_RST = 1

7.3.2 HLT Instruction

HLT instruction **(Opcode 0x1)** is just used to halt the processor indefinitely.

$T_3 \rightarrow$ HALT = 1

7.3.3 JMP Instruction

The **JMP** instruction opcode **(0x8)** is one of the most important control flow instructions. It can be used to unconditionally return the Processor to a previous state in a loop with no exit conditions

$T_3 \rightarrow$ PC_OUT = 1; MAR_IN = 1;
$T_4 \rightarrow$ PC_WR = 1; IMEM_RD = 1; STP_RST = 1

We don't need to increment the PC as it runs in advance.

7.3.4 JP Instruction

The **JP** instruction opcode **(0xE)** is similar to JMP but it extracts its destination address from Rd.

$T_3 \rightarrow$ REG_A_RD = 1; PC_WR = 1; STP_RST = 1

7.3.5 JZ, JC Instructions

Opcodes **0x9**, **0xA** maps to **JC** and **JZ** respectively; these are conditional JUMP, they look at previous arithmetic state of the ALU and determine Jump.

7.3.5.1 Conditional Branching In JZ and JC

JC (Jump If Carry) and **JZ** (Jump If Zero), use conditional early exit branching to reduce cycles. In case the conditions are met they take 4 machine cycles, early exit branching helps in exiting early if conditions are not met taking only 3 machine cycles. That saves us 25% of CPU cycles approximately 90% of the time.

The **JZ** and **JC** instructions provide the STEPLA-1 with its logical decision-making capability. To maintain a low gate count and bypass the "Hardwired CAP," these instructions **borrow hardware paths** from the Unconditional **JUMP** circuitry.

v2.4.0 introduces 'Early-Exit Branching,' a dynamic state-machine optimization that detects failed conditions at T_3. By compressing the instruction stream and removing the 1-cycle from every operation, we have achieved a 30% throughput increase.

The "Fibonacci" Efficiency Case Study

In real-world iterative algorithms, conditions are statistically "Not Met" more often than "Met."

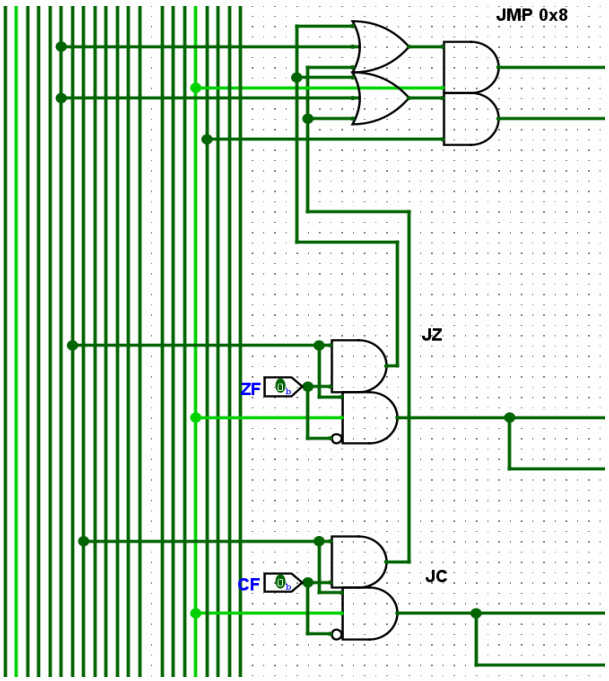
- Example: In an 8-bit Fibonacci sequence, the JC (Jump if Carry) condition is Not Met 91.6% of the time (11 out of 12 iterations).
- Optimization Impact: By exiting at T_3, the STEPLA-1 eliminates the "Instruction Tax" on loops. In a standard Fibonacci run, this optimization saves 33 total clock cycles, representing a 25% reduction in branch latency for the majority of the program's runtime.

Hardware Implementation Note: Conditional Path Logic

The STEPLA-1 utilizes **shared control lines** between the Unconditional JUMP and the Conditional JZ/JC instructions to minimize gate density. The logic is handled by 3-input AND gates with inverted (NOT) inputs for the flag states.

- **Condition Met (True):** If the flag is active, the PC_INC and STP_RST signals are suppressed during T_3. The instruction is allowed to proceed into T_4 and T_5, where the "Heavy Lifting" is handed over to the standard JUMP hardware to load the target address.
- **Condition Not Met (False):** If the flag is inactive, the inverted input on the AND gate triggers an **Immediate Reset**. In this single machine cycle, the hardware simultaneously pulses PC_INC (to point to the next instruction) and STP_RST (to return to Fetch).

Architectural Insight: The timing signals on the left of the schematic are shared between the *Start* of a JUMP and the *End* of a JZ/JC. This "Control Path Overlap" is the key to the STEPLA-1's high-speed conditional branching.



7.3.6 GETPC

Opcode **0xD**, is used to take a snapshot of the computer address state and store it to a register. It can help the processor use JP to come back to this state address.

Functional Purpose

In a computer system, the Program Counter always points to the memory address of the next instruction to be executed. The `GETPC` instruction allows the processor to take a "snapshot" of this address. This is a critical feature for implementing **Subroutines** (functions) and **Relative Branching**.

By saving the current address into a register before performing a **`JMP`** (Jump), the processor creates a "Return Address." Once the sub-task is completed, the processor can use that stored address to jump back to exactly where it left off, ensuring the main program flow continues without interruption.

7.4 Arithmetic Instructions

7.4.1 ADD

Opcode 0x2, can add contents of Rs, Rd and store the result to Rd.

T_3 → REG_B_RD = 1; ACCUM_A_WR = 1;
T_4 → REG_A_RD = 1; ACCUM_B_WR = 1
T_5 → ALU_OUT_EN = 1; FL_EN = 1; REG_A_WR = 1; STP_RST = 1

7.4.2 SUB

Opcode 0x3, can subtract contents of Rs from Rd and store the result to Rd (Rd - Rs).

T_3 → REG_A_RD = 1; ACCUM_A_WR = 1;
T_4 → REG_B_RD = 1; ACCUM_B_WR = 1
T_5 → ALU_OUT_EN = 1; FL_EN = 1; REG_A_WR = 1; SUB = 1; STP_RST = 1

7.5 Output

7.5.1 OUT

Shows the stored values of any register on an 8 bit to 7 segment display.

T_3 → REG_A_RD = 1; OUT_REG_EN = 1; STP_RST = 1

---

8. Control Unit Sequencing and Clock Phase Synchronization

The **STEPLA-1** architecture utilizes a multi-phase clocking strategy to resolve critical timing hazards inherent in synchronous bus-based systems. The following protocols govern the interaction between the Step Counter, the instruction fetch cycle, and the external bootloader interface.

8.1 Phase Separation and Propagation Delay

To ensure data integrity, the STEPLA-1 enforces a strict half-cycle separation between datapath updates and control sequencing.

- **Datapath Operations:** All general-purpose registers and the Program Counter (PC) are triggered on the **Rising Edge** of the clock.
- **Control Sequencing:** The Step Counter is triggered on the **Falling Edge** of the clock.

This 180° phase shift provides a necessary stabilization window. By delaying the transition of the Control Unit's state until the falling edge, the architecture guarantees that all electrical signals on the bus have reached a stable logic level following the rising-edge "muscle" movement of the registers. This prevents the execution of "meta-stable" or incomplete data.

8.2 The T0 Null State and Bootloader Handover

The Control Unit is hard-wired to treat **T0** as a "Null State." During this cycle, no control lines are asserted, and the system remains in a passive state.

This design is a mandatory requirement for stable handovers from the system **Bootloader**. When the Bootloader releases the system "Lock" signal on a rising clock edge, the Step Counter immediately detects the subsequent falling edge. This transition causes the counter to bypass or briefly occupy T0. Without the Null State, the processor would attempt to execute the address currently residing in the **Memory Address Register (MAR)**; which, at the moment of handover, contains the final address used by the Bootloader. By designating T0 as an empty cycle, the STEPLA-1 prevents an unauthorized memory fetch and ensures the processor begins execution only when the fetch cycle formally initiates at T1.
Bootloader is further explained in the next section .

8.2.1 Timing Diagram for BCU handover (delay of 50ns included)

Timing diagrams and associated analysis are derived under **worst-case operating conditions** in order to establish a safety margin (threshold) that must not be exceeded for stable computer operation. This margin ensures reliable synchronization between control and datapath components during transitional phases of execution.

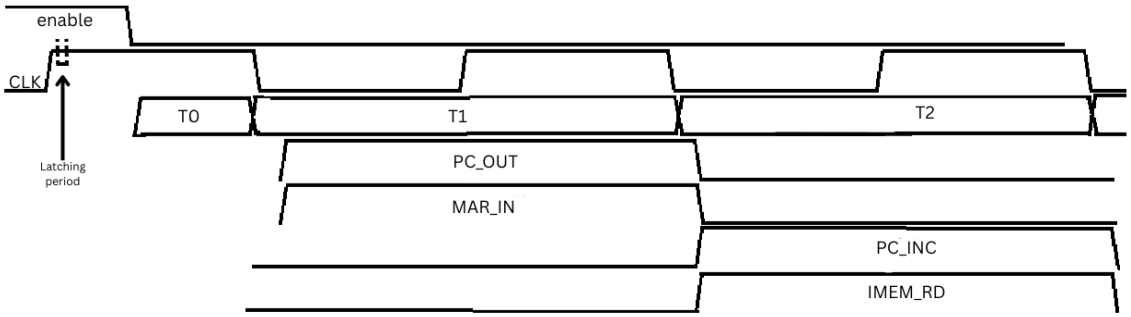
The presented diagrams include the system clock, the Control Unit enable signal (HALT), and relevant control signals such as **MAR_IN**, **PC_OUT**, **IMEM_RD**, and **PC_INC**. Execution states, including **T0**, are explicitly illustrated to clarify their role in stabilizing the STEPLA-1 system immediately following BCU handover.

As described previously, the step counter increments on clock intervals opposite to those used for register latching. This introduces an effective **180° phase shift**, allowing safe data propagation and preventing race conditions during register updates and control transitions.

During the handover phase, the enable signal shown in the figure transitions low only after approximately **one quarter of a clock cycle** has elapsed. This delayed transition effectively removes the available latching window for that cycle. As a consequence of the late handover, the **T0 state becomes short-lived** and advances to **T1** at the next falling edge of the clock.

Because insufficient time exists for the Memory Address Register (MAR) to latch data from the Program Counter (PC), the subsequent fetch cycle incorrectly accesses the last address previously used by the BCU.

Introducing a **null (extended) T0 state** resolves this issue by ensuring that the MAR successfully captures the new address from the PC. As shown in the timing diagram, **PC_OUT** is provided with approximately half a clock cycle to stabilize before being sampled by MAR on the following clock high edge. This guarantees that valid address data is latched prior to the next step counter increment, thereby restoring correct fetch-cycle behavior.



Timing Graph .1

8.3 Instruction Fetch and Latching (T1–T3)

The instruction is distributed across three specific cycles to ensure reliable opcode capture:

1. **T1 Cycle:** The Address Bus is populated with the Program Counter value.

2. **T2 Cycle:** The Opcode Register (Instruction Register) latches the data from memory.
3. **T3 Cycle:** The Control Unit decodes the latched opcode and initiates the execution phase.

Earlier iterations utilized an idle T2 state to prevent Step Counter resets before the Opcode Register could latch data. In the current STEPLA-1 specification, the combination of the falling-edge Step Counter and the T1-T2 fetch window ensures that the opcode is securely locked into the instruction register before the decoder evaluates the command at T3. This architecture eliminates the risk of "skipping" or misinterpreting instructions during high-frequency operation or system resets.

---

9. Bootstrap Control Unit (BCU) Specification

9.1 Functional Overview

The **Bootstrap Control Unit (BCU)** is a dedicated hardware sub-system responsible for the non-volatile initialization of the STEPLA-1 processor. Upon system power-on or a hardware reset, the BCU performs a "Cold Boot" procedure, which involves migrating machine-code instructions from an onboard Read-Only Memory (ROM) to the system's primary Random Access Memory (RAM).

The primary purpose of the BCU is to resolve the initial execution paradox: the Central Processing Unit (CPU) requires a valid instruction stream to operate, yet cannot manually populate the volatile RAM without an existing program. The BCU acts as a bridge, preparing the execution environment before handing control to the Main Control Unit.

9.2 Bus Arbitration and CPU Suspension

The BCU operates as a **Bus Master** with higher priority than the CPU.

1. **CPU Halting:** Upon activation, the BCU asserts a global HALT signal. This signal suspends the Main Control Unit and effectively disconnects the CPU's registers from the Address and Data buses via high-impedance (Tri-state) isolation.
2. **Bus Acquisition:** While the CPU is in a suspended state, the BCU gains exclusive access to the Memory Address Register (MAR) and the RAM write-control circuitry.

9.3 The Dual-Cycle DMA Protocol

To efficiently transfer data over a shared 8-bit architecture, the BCU utilizes a **two-cycle state machine** synchronized by a JK-type Flip-Flop. This alternating logic prevents bus contention and ensures data integrity.

Phase A: Address Latching (State 0)

During the first cycle, the BCU focuses on the "Where."

- The JK-Flip-Flop is set to 0.
- The BCU drives the target memory address from its internal counter onto the system bus.
- A load pulse is sent to the **Memory Address Register (MAR)**.
- **Result:** The system memory is now indexed to the correct location for the incoming byte.

Phase B: Data Transfer (State 1)

During the second cycle, the BCU focuses on the "What."

- The JK-Flip-Flop toggles to 1.
- The BCU retrieves the instruction byte from the Bootstrap ROM and drives it onto the Data Bus.
- A **RAM Write Enable (WE)** pulse is generated.
- **Result:** The instruction is successfully written to the RAM address specified in Phase A.

9.4 Release and Synchronization (The T0 Transition)

Following the completion of the ROM-to-RAM transfer, the BCU enters the **Handover Phase**.

1. **Lock Release:** The BCU de-asserts the HALT signal on a rising clock edge.
2. **Phase Stabilization:** Due to the hardware design, the release of the lock triggers the falling-edge sensitive Step Counter.
3. **The T0 Null State:** Because the MAR still contains the final address utilized by the BCU, the STEPLA-1 is programmed to treat its first cycle (**T0**) as an idle/unused state. This prevents the processor from fetching data from the BCU's last-accessed memory location.
4. **Execution Start:** The Main Control Unit formally initiates its first instruction fetch at **T1**, at which point the Program Counter (PC) has regained control of the Address Bus.

9.5 Technical Constraints

- **Clocking:** The BCU logic is synchronized with the primary system clock but operates independently of the CPU instruction cycle.
- **Bus Isolation:** Failure to maintain high-impedance states on CPU buffers during BCU operation will result in bus contention and data corruption.
- **Memory Mapping:** The BCU is hard-coded to populate RAM starting at address 0x0.
- **Exit Conditions:** The BCU is hard coded to exit on either one of conditions, if the counter has overflowed or if the ROM’s last address data maps to **0xff** or **0b11111111.**

10. Control Logic Generation

10.1 Core Principle: The Matrix Decoder

The STEPLA-1 utilizes a hard-wired logic matrix to derive control signals. The fundamental operation of the Control Unit is to synchronize a specific **Instruction** with a specific **Time Step**.

The generation of any control signal follows a strict Boolean "AND" relationship:

Control Signal = Instruction Decoded Line & Active Timing Step

This ensures that a signal (such as a register load) only fires when the processor is in the correct state and executing the correct command.

Example: Load Accumulator (LDA) Execution
To execute an ADD command, the Accumulator register must be enabled to receive data from the bus. This must happen only during the specific execution step (e.g., **T3**) of that specific instruction.

- **Logic:** ACCUM_A_WR = ADD_ACTIVE & STEP_T3

10.2 Signal Consolidation (The OR-Bus)

Many instructions share the same hardware actions. For example, both an **ADD** instruction and a **SUB** instruction need to enable the ALU output onto the bus.

To handle this, the STEPLA-1 uses **OR Gates** to merge multiple instruction-specific signals into a single hardware control line.

Special Signals like **JZ** and **JC** directly utilize **JMP** instruction hardware if the conditions are right. In case they are not triggered, the step counter is reset in the next cycle.

---

11. Timing Analysis & Operational Limits

The transition from a simulated environment (Logisim) to a physical breadboard implementation requires a rigorous analysis of **Asynchronous Propagation Delays** and **Signal Integrity**. While Logisim treats wires as ideal conductors, a physical STEPLA-1 build using the **74LS (Low-Power Schottky)** series must account for the electrical characteristics of discrete logic.

11.1 Real-World Timing Constraints

This section evaluates the operational limits of the STEPLA-1 by analyzing four critical hardware factors:

1. **Gate Propagation Delay (t_{pd}):** The cumulative time required for a signal to traverse the Opcode and Step decoders before reaching the Control Matrix.
2. **Memory Access Latency (t_{acc}):** The 150ns bottleneck imposed by the AT28C64 EEPROM during the Instruction Fetch phase.
3. **Fan-out & Capacitive Loading:** The degradation of signal **Rise Time** t_r when a single logic output (e.g., Step T1) drives multiple register inputs.
4. **Parasitic Inductance:** The "ringing" and noise caused by long jumper wires on a breadboard, which can lead to false triggering or metastability.

11.2 Component Latency Profiles

To establish a **speed ceiling** for bread-board implementation, we will need to account for worst case latency of 74LS series and memory access latency for ROM and RAM from provided Data Sheets.

Component Categ.	Typical Part	t_{pd} (Max)	Role
Combinational Logic	74LS08/32	110ns	
Storage	74LS377	40ns	

// ROM and RAM have a t_{pd} of 150 to 70 ns. Write wait t_{min} is 10 ns response can range from 100 ns to 1000 ns based on the type of EEPROM used.

12. Signal Conditioning and Bus Termination

Traditional TTL logic and breadboard-based processors often struggle with "sourcing" current (driving a signal to 5V) against the inherent resistance and capacitance of long jumper wires. The STEPLA-1 architecture utilizes **Active-Low Signaling** to leverage the superior current-sinking capabilities of logic ICs. By maintaining a high voltage state as the "idle" default, the system effectively "pre-charges" parasitic capacitances, ensuring the bus is ready for high-speed transitions.

12.1 Active-Low Logic & Capacitive Displacement

By inverting the inputs and outputs of every major component, the architecture adopts a **"0 = ON / 1 = OFF"** logic convention.

- **Physics of the Pull-Down:** While charging a wire to 5V (Active-High) can be slowed by parasitic capacitance, "pulling down" a pre-charged wire to Ground (0V) is nearly instantaneous.
- **Sharp Transitions:** This method keeps the bus "filled" with potential energy at all times. When a signal is transmitted, the sudden drop to 0V creates a sharp, vertical falling edge that requires minimal settling time, allowing for significantly higher clock frequencies without data corruption.

12.2 Schmitt Trigger Conditioning (74HC14)

Standard **NOT gates** behave strangely to ambiguous signals; they transition with voltages that are stable near **5V or 0V**. That makes them prone to glitches against interference and voltages below **2.5V** or above **5V**.

To further immunize the system against electromagnetic interference (EMI) and signal "ringing," the STEPLA-1 utilizes **Schmitt Trigger Inverters** (specifically the **74HC14**) for all critical signal paths.

- **Hysteresis as a Filter:** Unlike standard NOT gates that flip at a single, unstable threshold, Schmitt triggers employ **Hysteresis**.
- **Threshold Logic:** They require the signal to drop below a low threshold (typically **~1.5V**) to register a "Logic 0" and rise above a high threshold (typically **~3.0V**) to register a "Logic 1."
- **The "Dead Zone":** Any noise, vibration, or "mushy" voltage levels sitting between these two points are ignored by the component. This makes the processor "deaf" to the electrical turbulence common on breadboards, enabling stable operation at speeds that would typically cause a standard discrete-logic computer to crash.

12.3 Bus Pull-up Networks & Passive Termination

To support the Active-Low architecture, the STEPLA-1 utilizes a Passive Pull-up Network on all primary bus lines.

- **The Idle State:** Resistors (typically 4.7kΩ to 10kΩ) are tied between the bus lines and VCC (5V). This ensures that when no component is active, the bus is held at a rock-solid Logic 1.
- **Eliminating "Floating" Errors:** This prevents the high-impedance (tri-state) mode of the chips from picking up environmental noise the "ghost signals" that often cause crashes in solo-built computers.

---

13. Design Decisions

- **Harvard vs. Von Neumann Influence:** While the system utilizes a shared memory space, the **Bootstrap Control Unit** acts as a temporary Harvard-style loader, ensuring that code is treated as immutable data during the boot phase before becoming executable. The presence of separate Data SRAM also ensures this architecture
- **Dual-Phase Clocking:** The decision to use **Rising Edge** for datapath registers and **Falling Edge** for the Step Counter was made to eliminate race conditions. This provides a 50% clock-cycle "settling time" for electrical signals, prioritizing stability over maximum theoretical clock speed.
- **Hard-Wired Logic over Microcode:** To keep the architecture transparent for educational purposes, a combinatorial logic matrix was chosen over a ROM-based microcode engine. This allows for direct tracing of signals through the AND/OR gate arrays.
- **RISC based Instructions:** While the ISA was designed independently, some features were inspired too. RISC was one of them, increasing the Program Counter in advance after every fetch is one of them, this ensures smooth transitions and jumps without a need to calculate and ensure the presence of an offset using NOP or any other hardware component.

---

14. Verification Plan

To ensure the reliability of the hardware logic, the following testing protocols are implemented:

- **The recursive Stress Test:** A recursive addition program (1+2+3...) is used to verify ALU flags (Zero and Carry) and the stability of the Jump (JZ) logic.
- **Bootloader Integrity Check:** Verification that the BCU correctly populates RAM address 0x0 and successfully releases the bus without "poisoning" the first CPU fetch.
- **Clock-Speed Torture Test:** Incrementally increasing the simulation frequency in Logisim to identify the exact point where propagation delay exceeds the half-cycle "Safety Gap." (**Note: It was observed that Logisim Evolution is unable to simulate frequencies more than 256 Hz, however calculations show that failure attributed to high frequencies can occur at around 2 - 5 Mhz**).

  // Note: Introduce test vectors

---

15. Known Limitations

Simulation Lag: Due to the high gate count of the 8-bit multi-core expansion, the Logisim simulation may experience a drop in "Ticks per Second," requiring a move to high-performance simulators (like Digital) for complex software.
Lack of Hardware Stack: Currently, subroutines must be managed manually using registers, as a dedicated Hardware Stack Pointer has not yet been integrated into the Control Unit logic, It uses GETPC to save a snapshot of the Processor and the computer can come back to this state itself useful for nested functions and Sub-routines.
Sequential Fetching: The current design does not support instruction "Pipelining." Each instruction must fully complete its T-states before the next fetch begins, limiting the Instructions Per Clock (IPC) to less than 1.0.

---

16. Future Architectural RoadMap (v3.0.0)

16.1 Multi-Cycle Instruction Fetching

The primary objective for the next iteration of the **STEPLA-1** is to bypass the addressing limitations of a single-byte instruction format. By implementing a **Multi-Cycle Fetch (MCF)** mechanism, the architecture will transition to a **16-bit instruction word** while retaining the existing **8-bit memory bus**. This requires a modification to the Step Decoder to include a secondary fetch state, allowing the Control Unit to pull an "extended opcode" or "immediate operand" from the subsequent memory address before entering the execution phase.

16.2 Expansion of Register File and Opcode Space

This 16-bit expansion is designed to solve two critical bottlenecks identified in v2.2.2:

- **Register Addressing:** The current design is limited by the bit-width of the opcode. A 16-bit format allows for a **4-bit Source/Destination encoding**, enabling a transition from the current minimal register set to a **16-register General Purpose Register (GPR) file**.
- **Instruction Complexity:** The available opcode space will expand from 2^8 to 2^{16}, facilitating the inclusion of complex instructions such as **Relative Branching**, **Bitwise Rotations**, and **Multi-word Arithmetic**, which were previously omitted due to space constraints.

16.3 Hardware-Level Stack Management

Future iterations will explore the implementation of a dedicated **Stack Pointer (SP)** register and associated control logic (PUSH, POP, CALL, RET). This will move the architecture away from manual link-register subroutines (via GETPC) toward a more robust, nested function call model, significantly enhancing the CPU's ability to handle complex software algorithms and recursive logic.

16.4 Asynchronous Dual Control Unit System

V3.0.0 will implement an Asynchronous Dual Control Unit system that will use two control units running concurrently, divided as Primary and Secondary Units. While Both Units will share a Stack with **mutual PUSH** and **exclusive POP**, they will use interrupts to PUSH and a FLUSH signal for primary CU to discard the next instruction Pipeline and and fetch a new instruction. Program Counter will be common between the two systems.

16.5 Advanced Interrupt Vectoring and Priority

The STEPLA-1 v3.0 will implement a **Dual-CU Interrupt Controller**. To ensure system stability during parallel operations, a hardware-level priority arbiter will be established.

- **First-Come-First-Served (FCFS) Arbitration:** If simultaneous interrupts occur, the arbiter grants control based on the arrival of the rising edge of the interrupt signal.

- **CU Assignment:** Interrupts can be routed to either the Primary or Secondary CU. For example, the Secondary CU can handle I/O-related interrupts in the background, allowing the Primary CU to maintain the "Main Loop" execution without a context switch, effectively reducing total system latency.

16.6 Pre-Fetch Buffer and Cycle Reduction

To maximize the efficiency of the 8-bit bus, the Secondary CU will act as a **Pre-fetch Engine**.

- **Mechanism:** While the Primary CU is occupied with an execution cycle, the Secondary CU utilizes these cycles to fetch the next opcode from RAM and store it in a **Next Instruction Register (NIR)**.
- **Efficiency Gain:** This eliminates the T_1 and T_2 fetch cycles for the Primary CU in the subsequent instruction. By overlapping the "Execute" phase of instruction with the "Fetch" phase of instruction N+1, the architecture approaches a theoretical **1.0 CPI (Cycles Per Instruction)** even with complex, multi-byte operations

---

17. Inspirations & Learning Material

16.1 Architectural Influences

- **The Ben Eater 8-Bit Computer:** A primary inspiration for the clock-cycle logic and the use of the 74-series logic gates philosophy. The STEPLA-1 adopts the "breadboard-ready" mentality of visible data flow.
- **The MOS 6502 ISA:** The instruction set design (ISA) borrows heavily from the 6502's register-to-register philosophy, specifically the use of an Accumulator and simple status flags for branching.
- **SAP-1 (Simple-As-Possible) Computer:** The foundational architecture described in Albert Malvino's *Digital Computer Electronics*.
- SRAM and BCU was inspired by "leonicolas/computer-8bits: A basic 8-bits computer created with LogiSim digital circuit simulator :computer:"

16.2 Recommended Reading

- **"Digital Computer Electronics" by Albert Malvino:** The "basis" for understanding how a simple Control Unit and Instruction Decoder function.
- **"Code: The Hidden Language of Computer Hardware and Software" by Charles Petzold**
- **Logisim-Evolution Documentation:** For understanding the propagation delays and simulation constraints involved in building the STEPLA-1.

- **“Computer Organization and Design" by David A. Patterson & John L. Hennessy**

Special Thanks to Leon Nicolas